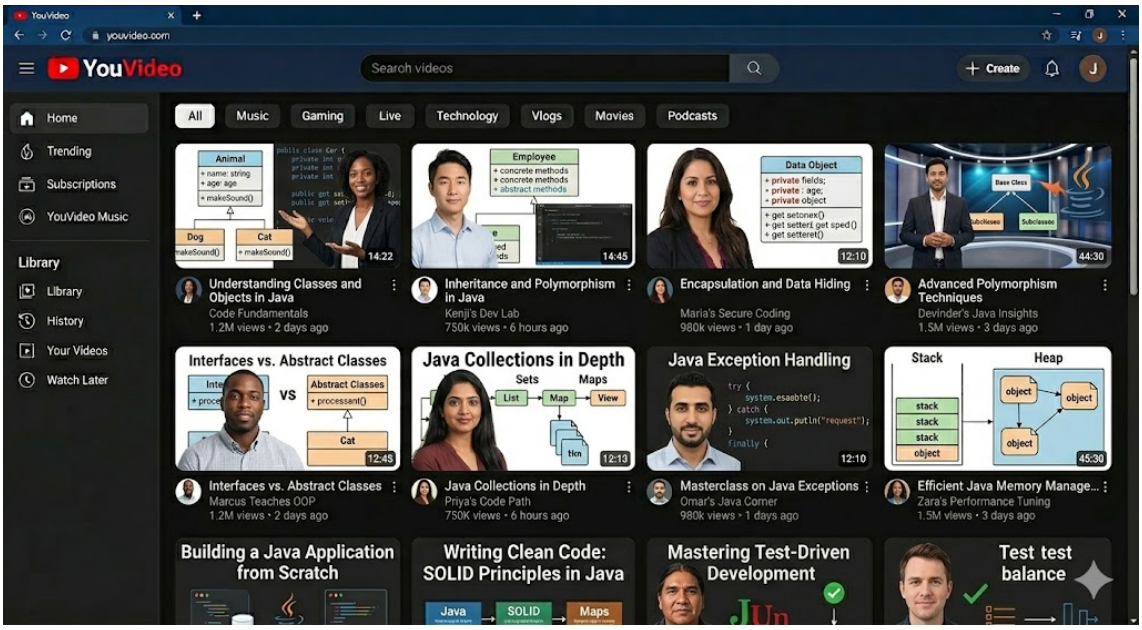
YouVideo

Object-Oriented Programming Project Phase 1
version 1.0 – 2026-03-24



Important remarks

Deadline until 17h00 - 5PM (Lisbon time) of April 26th, 2026. Submissions are only possible via Mooshak. Late submissions will suffer a penalty of 1 point (out of 20) for each set of 8 hours of delay (for example, a submission delivered 10 hours after the deadline will suffer a 2-point penalty).

Project Submission Information This project will be submitted in two phases, allowing you to progressively enhance your solution as you acquire new knowledge and skills throughout the semester. This document describes the requirements for the first phase submission, which focuses on applying the object-oriented programming concepts and techniques covered up to March 25th. For this initial phase, you should model the domain to the best of your current ability, demonstrating your understanding of the principles of object oriented design, extensibility, encapsulation, inheritance, polymorphism, interfaces, abstract classes and generic types.You are expected to create a clean object-oriented model that captures the entities and their relationships as described, using the package structure and access control mechanisms discussed in class. Remember that all code should adhere to the quality standards that we have established, including declaring variables using interfaces and storing user-facing strings as constants.

In the second phase, you will have the opportunity to refine and improve your solution based on the additional Java collections framework concepts and interfaces you will learn from this point forward. You will also need to address new requirements that will be

added to the problem statement, potentially introducing more complex relationships or functionality. This phased approach mirrors real-world software development, where requirements evolve and designs are refined over time. By the end of the project, you will have experienced the full lifecycle of building a domain model, from initial object-oriented design to leveraging the power of Java's built-in collections and APIs, resulting in a more robust and maintainable solution.

Team This project is to be MADE BY GROUPS OF 2 STUDENTS.

Deliverables: Submission and acceptance of the source code to **Mooshak** (score > 0). See the course website for further details on how to submit projects to Mooshak. Your code should be documented according to Javadoc rules and must include detailed implementation comments where relevant. The submission should also include UML diagram of your interface and class hierarchy.

Recommendations: We value the documentation of your source code, as well as the usage of the best programming style possible and, of course, the correct functioning of the project. Please carefully comment both interfaces and classes. The documentation in classes can, of course, refer to the one in interfaces, where appropriate. **For this phase,** please comment on the methods explaining what they mean and defining preconditions for their usage. Students may and should discuss any doubts with the teaching team, and with other students, but may not share their code with other colleagues. This project is to be conducted with full respect for the Code of Ethics of NOVA University, Lisbon, available on the course website.

1 Development of the application *YouVideo*

The way we consume video content has been transformed by streaming platforms like Netflix, YouTube, and Spotify. These services offer vast libraries of content, from user-created videos to premium productions, often accompanied by features like subtitles in multiple languages, episode-based podcasts, and scheduled programming. In this project, you will develop a simplified video streaming platform that manages different types of video content, podcasts, and shows, capturing the essential relationships and constraints that exist in real-world streaming services.

1.1 Problem description

The core element of the system is video content, stored and managed in the system. Each **video** is characterized by a **unique identifier**, a **duration**, and a **file location**. The system stores **publishable videos**, which can be independently published, as well as podcast **episodes**, which can only be added to the system as part of independent podcasts. In addition to the core video information, publishable videos include a **title**, **publisher**, and **language**. Publishable videos are published by **basic** and **premium** subscribers. Videos published by premium subscribers are called **premium** videos and, additionally to the information included in Publishable videos, may include one or more subtitles. **Subtitles** themselves are defined by a language and a **subtitle file**, and can be added to premium videos over time. Premium videos will thus include a list of its subtitles, ordered by the moment of insertion into the list. More than one subtitle of the same language may be added to a premium video. The system must ensure that **each video maintains a unique identifier** across the entire platform, regardless of its type.

Beyond individual videos, the platform also manages **podcasts** as a distinct form of content. A podcast is defined by its **title**, **author**, and **language**, and contains a **collection of video episodes** that are released over time. The title of a podcast must be unique in the complete collection of podcasts. A podcast is created with no episodes. A podcast episode is an unpublishable video, with no language information (as the language is provided in the podcast information). Each **episode** is thus a video with a **release date**. Episodes within a podcast must be organized in reverse chronological order, with the most recent episode appearing first. When adding a new episode, the system must enforce that its release date is not earlier than the most recent existing episode, ensuring chronological consistency. Episodes, being videos, must also have unique identifiers across the entire system and cannot exist independently of their parent podcast. Therefore, when removing a podcast, the system must remove all its episodes.

The platform also includes **shows**, which represent scheduled transmissions of publishable videos. A show's information includes an **author**, a **transmission date**, and a reference to a publishable video. Shows must have unique titles, and removing a show should not affect the underlying video content.

Throughout the system, you must maintain **appropriate collections** that allow efficient retrieval: **videos** organized by their **identifier**, **podcasts** sorted by **title**, and **shows** sorted by **title**. Additionally, the system must track **podcast authors** to support queries about an author's body of work.

The resulting application will be accessed through a **command-line interface** that allows users to **create, query, list and remove content** while respecting all business rules and constraints that define a coherent streaming platform.

1.2 Data issues

The data attributes to be used in the system will include:

- **Integer values**, larger than zero, to be used for video durations;
- **One word strings**, to be used for video identifiers, language codes (two letter strings), dates(YYYY-MM-DD) and URLs and filenames (long one word strings);
- **Strings including more than one word**, for video and show titles, as well as publisher and author names.

The set of possible languages to be used in this project is defined in the **ISO 639-1 language standard**[1] and can be accessed in the `java.util.Locale` class, through the method `getISOLanguages()`. Each language value can be stored as `Locale` objects which may be created using two-letter strings. For this you can use the `Locale(String l)` constructor (e.g. `Locale lang = new Locale("ES")`). To access the display name of the language you can use the `getDisplayLanguage()` method (e.g. `lang.getDisplayLanguage()`).

2 Commands

In this section, we present all the commands that the system must be able to interpret and execute. In the following examples, we differentiate `text written by the user` from the feedback written by the program in the console. You may assume the user will make no mistakes when using the program other than those described in this document. In other words, you only need to take care of the error situations described here, in the same order as they are defined.

[1] https://localizely.com/iso-639-1-list/

Commands are case-insensitive. For example, the `exit` command may be written using any combination of upper and lowercase characters, such as `EXIT`, `exit`, `Exit`, `exIT`, and so on. In the examples provided in this document, the symbol ↵ denotes a line change.

The program must write the message Unknown command. Type help to see available commands. to the console if the user introduces an unknown command. For example, the non existing command `someRandomCommand` would have the following effect:

```
someRandomCommand↵
Unknown command. Type help to see available commands.↵
```

If there are additional tokens in the line (e.g., a parameter for the command you were trying to write), the program will also try to consume them as commands. So, in the following example, `someRandom Command` would be interpreted as two unknown commands: `someRandom` and `Command`, leading to two error messages.

```
someRandom Command↵
Unknown command. Type help to see available commands.↵
Unknown command. Type help to see available commands.↵
```

Several commands have arguments. Unless explicitly stated in this document, you may assume that the user will only write arguments of the correct type. However, some of those arguments may have an incorrect value. Therefore, we must test each argument precisely in the order specified in this document. In their description, arguments will be denoted **`with this style`**, for easier identification. Also, assume by default that any String arguments in the commands are case-insensitive. So, "Java One course" and "java ONE COURSE" would refer to the title of the same video .

2.1 `exit` command

Terminates the execution of the program. This command does not require any arguments. The following scenario illustrates its usage.

```
exit↵
Bye!↵
```

This command always succeeds.

2.2 `help` command

Shows the available commands. This command does not require any arguments. The following scenario illustrates its usage.

```
help↵
createpublishable - creates a new publishable video↵
createpremium - creates a new publishable Premium video↵
addsubtitle - adds subtitle to Premium video↵
getvideo - presents publishable video data from its id↵
subtitles - Lists Premium video subtitles↵
createpodcast - creates a new podcast with no episodes↵
addepisode - adds an episode to a podcast↵
getpodcast - presents podcast data from its title↵
episodes - List podcast episodes↵
authorpodcasts - List all podcasts of an author↵
removepodcast - removes a podcast↵
createshow - creates show using an existing publishable video↵
getshow - presents show data from its title↵
removeshow - removes a show↵
removevideo - removes a publishable video↵
help - shows the available commands↵
exit - terminates the execution of the program↵
```

This command always succeeds. When executed, it shows the available commands.

2.3 `createpublishable` command

Creates a new publishable video. This command creates a new publishable video in the system. The command requires six parameters: a unique string video identifier (**`id`**), the video **`duration`** in minutes, the **`URL`** where the video file is stored, the **`publisher`** name, the video **`title`**, and a two-character string language **`code`** indicating the primary language of the video content. The **id must be unique** across all videos in the system.

```
createpublishable vid440 250 http://teste.pt/PHPCourse.wav↵
CourseMaster↵
PHP Course↵
EN↵
Video vid440 created successfully.↵
```

The following errors may occur:

1. If the language **`code`** is invalid, the error message is (Invalid language type.).
2. If the **`duration`** is invalid, the error message is (Invalid value.).
3. If the system already has a video with the given **`id`**, the error message is (Video with this ID already exists.).

The following example illustrates these error messages.

```
createpublishable vid440 250 http://teste.pt/PHPCourse.wav↵
CourseMaster↵
PHP Course↵
XX↵
Invalid language type.↵
createpublishable vid440 0 http://teste.pt/PHPCourse.wav↵
CourseMaster↵
PHP Course↵
EN↵
Invalid value.↵
createpublishable vid440 250 http://teste.pt/PHPCourse.wav↵
CourseMaster↵
PHP Course↵
EN↵
Video with this ID already exists.↵
```

2.4 `createpremium` command

Creates a new publishable premium video. This command creates a new premium video, a publishable video with subtitle support. The command requires eight parameters: a unique string video identifier (**`id`**), the video **`duration`** in minutes, the **`URL`** where the video file is stored, the **`publisher`** name, the video **`title`**, the primary language **`code`**, the **`subtitleUrl`** where the initial subtitle file is stored, and the **`subtitleLanguage`** code for that subtitle. The **id must be unique** across all videos in the system, and the video is created with at least one subtitle.

```
createpremium vid456 180 http://example.com/vid456.mp4↵
EduChannel↵
Advanced Java↵
EN↵
http://example.com/subtitles-en.vtt↵
EN↵
PREMIUM Video vid456 created successfully.↵
```

The following errors may occur:

1. If the primary language **`code`** is invalid, the error message is (Invalid language type.).
2. If the subtitle language **`code`** is invalid, the error message is (Invalid language type in subtitle.).
3. If the **`duration`** is less than or equal to zero, the error message is (Invalid value.).
4. If the system already has a video with the given **`id`**, the error message is (Video with this ID already exists.).

The following example illustrates these error messages.

```
createpremium vid456 180 http://example.com/vid456.mp4↵
EduChannel↵
Advanced Java↵
XX↵
http://example.com/subtitles-en.vtt EN↵
Invalid language type.↵
createpremium vid456 180 http://example.com/vid456.mp4↵
EduChannel↵
Advanced Java↵
EN↵
http://example.com/subtitles-en.vtt XX↵
Invalid language type in subtitle.↵
createpremium vid456 0 http://example.com/vid456.mp4↵
EduChannel↵
Advanced Java↵
EN↵
http://example.com/subtitles-en.vtt EN↵
Invalid value.↵
createpremium vid456 180 http://example.com/vid456.mp4↵
EduChannel↵
Advanced Java↵
EN↵
http://example.com/subtitles-en.vtt EN↵
Video with this ID already exists.↵
```

2.5 `addsubtitle` command

Adds a subtitle to an existing Premium video. This command adds a new subtitle to an existing premium video. It takes three parameters: the identifier of the premium video (**`videoId`**), the **`subtitleUrl`** where the subtitle file is stored, and the **`subtitleLanguage`** code. The system adds the subtitle to the video's collection, preserving the order of addition. Multiple subtitles in the same language are permitted.

```
addsubtitle vid456 http://example.com/subtitles-es.vtt↵
ES↵
Subtitle added successfully.↵
```

The following errors may occur:

1. If the subtitle language **`code`** is invalid, the error message is (Invalid language type in subtitle.).
2. If the **`videoId`** does not exist, the error message is (Video does not exist.).
3. If the video exists but is not a premium video, the error message is (This operation requires a Premium video.).

The following example illustrates these error messages.

```
addsubtitle vid456 http://example.com/subtitles-es.vtt↵
XX↵
Invalid language type in subtitle.↵
addsubtitle vid999 http://example.com/subtitles-es.vtt↵
ES↵
Video does not exist.↵
createpublishable vid123 120 http://example.com/vid123.mp4↵
TechChannel↵
Introduction to Java↵
EN↵
Video created successfully.↵
addsubtitle vid123 http://example.com/subtitles-es.vtt↵
ES↵
This operation requires a Premium video.↵
```

2.6 `getvideo` command

Presents video data from its ID. This command retrieves and displays all information about a video using its identifier. It requires a single parameter: the video identifier (**`videoId`**). The system then displays the video type (if it is PREMIUM), its ID, duration, title, file URL, publisher, and language.

```
getvideo vid123↵
Video vid123 120 Title: Introduction to Java↵
File: http://example.com/vid123.mp4 Publisher: TechChannel Language: ENGLISH↵
```

```
getvideo vid456↵
PREMIUM Video vid456 180 Title: Advanced Java↵
File: http://example.com/vid456.mp4 Publisher: EduChannel Language: ENGLISH↵
```

The following errors may occur:

1. If the **`videoId`** does not exist, or if the id belongs to a podcast episode the error message is (Publishable Video **`videoId`** does not exist.).

The following example illustrates this error message.

```
getvideo vid999↵
Publishable Video videoId does not exist.↵
```

2.7 `subtitles` command

Lists all subtitles of a Premium video. This command displays all subtitles associated with a premium video. It requires a single parameter: the identifier of the premium video (**`videoId`**). The system shows a header with the video's title, followed by each subtitle's URL and language code in the order they were added.

```
subtitles vid456↵
Subtitles for video Advanced Java:↵
- http://example.com/subtitles-en.vtt (ENGLISH)↵
- http://example.com/subtitles-es.vtt (SPANISH)↵
```

The following errors may occur:

1. If the **videoId** is not associated with a premium video, the error message is (No Premium Video with ID.). This error may occur whether the identifier does not exist in the system, or whether it exists but it references a non premium publishable video or an episode of a podcast.

The following example illustrates these error messages.

```
subtitles vid999↵
No Premium Video with ID.↵
subtitles vid123↵
No Premium Video with ID.↵
subtitles ep001↵
No Premium Video with ID.↵
```

2.8 `createpodcast` command

Creates a new podcast with no episodes. This command creates a new podcast in the system. It requires three parameters: the podcast **title**, the **author** name, and a two-character language **code**. The podcast title must be unique across all podcasts. The new podcast starts with an empty collection of episodes, ready to be populated over time.

```
createpodcast Tech Talk Daily↵
John Smith↵
EN↵
Podcast created successfully.↵
```

The following errors may occur:

1. If the language **code** is invalid, the error message is (Invalid language type.).
2. If a podcast with the same **title** already exists, the error message is (Podcast with this title already exists.).

The following example illustrates these error messages.

```
createpodcast Tech Talk Daily↵
John Smith↵
XX↵
Invalid language type.↵
createpodcast Tech Talk Daily↵
John Smith↵
EN↵
Podcast with this title already exists.↵
```

2.9 `addepisode` command

Adds an episode to a podcast. This command creates a new episode and adds it to an existing podcast. It requires five parameters: the podcast **`title`**, a unique episode **`id`**, the episode **`duration`** in minutes, the episode **`url`**, and the release **`date`** in YYYY-MM-DD format. The episode ID must be unique across the entire video system. The date is validated against the podcast's most recent episode: new episodes must have a date greater than or equal to the latest existing episode.

```
addepisode Tech Talk Daily↵
ep001 45 http://example.com/ep001.mp3↵
2025-03-15↵
Episode added successfully.↵
```

The following errors may occur:

1. If the **`duration`** is less than or equal to zero, the error message is (Invalid value.).
2. If the podcast **`title`** does not exist, the error message is (Podcast does not exist.).
3. If the episode **`id`** already exists in the system, the error message is (Episode ID already exists in the system.).
4. If the episode **`date`** is earlier than the latest episode date, the error message is (Episode date must be >= than latest episode date.).

The following example illustrates these error messages.

```
addepisode Tech Talk Daily↵
ep001 0 http://example.com/ep001.mp3↵
2025-03-15↵
Invalid value.↵
addepisode Unknown Podcast↵
ep001 45 http://example.com/ep001.mp3↵
2025-03-15↵
Podcast does not exist.↵
addepisode Tech Talk Daily↵
vid123 45 http://example.com/ep001.mp3↵
2025-03-15↵
Episode ID already exists in the system.↵
addepisode Tech Talk Daily↵
ep001 45 http://example.com/ep001.mp3↵
2025-03-01↵
Episode date must be >= than latest episode date.↵
```

2.10 `getpodcast` command

Presents podcast data from its title. This command retrieves and displays information about a podcast using its title. It requires a single parameter: the podcast **`title`**. The system displays the podcast's title, author, language, and, if episodes exist, the date of the

most recent episode. If the podcast has not transmitted any episodes, the command only prints the information about the podcast.

```
getpodcast Tech Talk Daily↵
Podcast: Tech Talk Daily Author: John Smith Language: EN↵
Latest episode date: 2025-03-15↵
```

The following errors may occur:

1. If the podcast **title** does not exist, the error message is (Podcast does not exist.).

The following example illustrates this error message.

```
getpodcast Unknown Podcast↵
Podcast does not exist.↵
```

2.11 episodes command

Lists all episodes of a podcast. This command displays all episodes of a podcast in reverse chronological order (newest first). It requires a single parameter: the podcast **title**. The system shows a header with the podcast title, followed by each episode's identifier, duration, date, and file URL.

```
episodes Tech Talk Daily↵
Episodes for podcast Tech Talk Daily:↵
Episode ep002: 50 min Date: 2025-03-22↵
URL: http://example.com/ep002.mp3↵
Episode ep001: 45 min Date: 2025-03-15↵
URL: http://example.com/ep001.mp3↵
```

The following errors may occur:

1. If the podcast **title** does not exist, the error message is (Podcast does not exist.).
2. If the podcast has no episodes, the error message is (No episodes available for this podcast.).

The following example illustrates these error messages.

```
episodes Unknown Podcast↵
Podcast does not exist.↵
episodes Empty Podcast↵
No episodes available for this podcast.↵
```

2.12 authorpodcasts command

Lists all podcasts by a specific author. This command finds and displays all podcasts created by a given author. It requires a single parameter: the **authorName**. The system shows a header with the author's name, followed by each podcast's title, author, and language. The podcasts should be listed by order of insertion in the system.

```
authorpodcasts John Smith↵
Podcasts by author John Smith:↵
Podcast: Tech Talk Daily Author: John Smith Language: EN↵
Podcast: Coding Adventures Author: John Smith Language: EN↵
```

The following errors may occur:

1. If the author has no podcasts in the system, the error message is (No podcasts found for this author.).

The following example illustrates this error message.

```
authorpodcasts Unknown Author↵
No podcasts found for this author.↵
```

2.13 `removepodcast` command

Removes a podcast by its title. This command completely removes a podcast and all its episodes from the system. It requires a single parameter: the podcast **`title`**. The system first removes all episodes belonging to the podcast from the videos collection, then removes the podcast itself. This cascading removal ensures that no orphaned episodes remain in the system.

```
removepodcast Tech Talk Daily↵
Podcast removed successfully.↵
```

The following errors may occur:

1. If the podcast **`title`** does not exist, the error message is (Podcast does not exist.).

The following example illustrates this error message.

```
removepodcast Unknown Podcast↵
Podcast does not exist.↵
```

2.14 `createshow` command

Creates a show using an existing publishable video. This command creates a new authored show, which will be broadcast an existing publishable video, on a particular transmission date. It requires three parameters: the show's **`author`**, the **`videoId`** of the video to be shown, and the **`transmissionDate`**. The publishable's video id must already exist in the system. The chosen video's title will be used to identify the show and must be unique in the context of the system's shows. This means that a show can only be created if the title of the video to be broadcasted, by the show, is unique in the collection of shows of the system.

```
createshow Jane Doe↵
vid123 2025-04-01↵
Show created successfully.↵
```

The following errors may occur:

1. If the **videoId** does not exist or does not reference a publishable video, the error message is (Video for show does not exist.).
2. If a show with the same title (as the chosen video's title) already exists, the error message is (Show with this title already exists.).

The following example illustrates these error messages.

```
createshow Jane Doe↵
vid999 2025-04-01↵
Video for show does not exist.↵
createshow Jane Doe↵
vid123 2025-04-01↵
Show with this title already exists.↵
```

2.15 getshow command

Presents a show's data, from its title. This command retrieves and displays information about a show using its title. It requires a single parameter: the show **title**. The system displays the transmission date, author, and the type and title of the video being shown.

```
getshow Introduction to Java↵
Show Date: 2025-04-01 Author: Jane Doe↵
Video: Introduction to Java↵
```

The following errors may occur:

1. If the show **title** does not exist, the error message is (Show does not exist.).

The following example illustrates this error message.

```
getshow Unknown Show↵
Show does not exist.↵
```

2.16 removeshow command

Removes a show, using its title. This command removes a show from the system without affecting the referenced video. It requires a single parameter: the show **title**. The video used in the show remains available in the system for other uses, reflecting the real-world practice of reusing content across multiple programming events.

```
removeshow Introduction to Java↵
Show removed successfully.↵
```

The following errors may occur:

1. If the show **title** does not exist, the error message is (Show does not exist.).

The following example illustrates this error message.

```
removeshow Unknown Show↵
Show does not exist.↵
```

2.17 `removevideo` command

Removes a video by its ID. This command removes a publishable video from the system using its identifier. It requires a single parameter: the video identifier (**`videoId`**). Episodes of a podcast cannot be removed with this command, they are not publishable videos. Videos that are part of shows cannot be removed.

```
removevideo vid123↵
Video removed successfully.↵
```

The following errors may occur:

1. If the **`videoId`** does not exist, the error message is (Video does not exist.).
2. If the video is an episode of a podcast, the error message is (Cannot remove: video is an episode of a podcast.).
3. If the video is used in a show, the error message is (Cannot remove: video is used in a show.).

The following example illustrates these error messages.

```
removevideo vid999↵
Video does not exist.↵
removevideo ep001↵
Cannot remove: video is an episode of a podcast.↵
removevideo vid123↵
Cannot remove: video is used in a show.↵
```

3 Developing this project

Your program should take the best advantage of the elements taught in the Object-Oriented Programming course. You should make this application as **extensible as possible** to make it easier to add, for instance, new kinds of video or program.

You can start by developing the main user interface of your program, clearly identifying which commands your application should support, their inputs and outputs, and the error conditions. Then, you need to identify the entities required for implementing this system. Carefully specify the **interfaces** and **classes** that you will need. You should document their conception and development using an interface and class diagram and document your code adequately with Javadoc. You can and should reuse the package on generic arrays presented in the lectures and labs (you can find a link to the source code on Moodle). You cannot change these classes and interfaces. If you need some slightly different functionality, you can either create classes that use those Arrays and iterators as instance variables and add, or constrain, the functionalities you need for your collections on those so-called wrapper classes. You can also consider extending those classes and interfaces. But do not change them. It is, in general, not a good idea to change library classes and interfaces. Also, please note that you cannot use the standard Java Collection classes in this project. Although they will be introduced in classes while you are still working on this project, we have designed this challenge for you to solve it without those classes. We will have ample opportunity of working with the standard Java Collection classes on the second phase and final submission of the project.

It is a good idea to build a skeleton of your `Main` class, to handle data input and output, supporting the interaction with your program. In an early stage, your program will not really do much. Remember the **stable version rule**: do not try to do everything at the same time. Build your program incrementally, and test the small increments as you build the new functionalities in your new system. If necessary, create small testing programs to test your classes and interfaces.

Have a careful look at the test files, when they become available. You should start with a really bare-bones system with the `help` and `exit` commands, which are good enough for checking whether your commands interpreter is working well, to begin with. Then, add publishable (and premium) videos and subtitles to premium videos. Once this is working you can create shows, which depend on the availability of videos. Check that they are ok. After that you can implement podcasts. Podcasts depend on extending the system with particular types of videos, episodes. Finally, you can implement the listing of the podcasts created by an author. Step by step, you will incrementally add functionalities to your program and test them. **Do not try to make all functionalities simultaneously. It is a really bad idea.**

Last, but not least, **do not underestimate the effort for this project. START EARLY**.

4 Submission to Mooshak

To submit your project to Mooshak, please register in the Mooshak contest POO2026-TP1 and follow the instructions that will be made available on the Moodle course website.

4.1 Command syntax

For each command, the program will only produce one output. The error conditions of each command have to be checked in the exact same order as described in this document. If one of those conditions occurs, you do not need to check for the other ones, as you only present the feedback message corresponding to the first failing condition. However, the program does need to consume all the remaining input parameters, even if they are to be discarded.

4.2 Tests

We expect you to create your own tests, based on this specification. Create tests for the "happy day scenario", and also for error situations, so that whenever you add support for a new command, you also create suitable tests for it. So, build your tests as you build your project, incrementally. This mimics what happens with the Mooshak tests. The Mooshak tests verify incrementally the implementation of the commands. They will be made publicly available roughly two weeks before the submission date. When the sample test files become available, use them to test what you already have implemented, fix it if necessary, and start submitting your partial project to Mooshak. Do it from the start, even if you just implemented the exit and help commands. By then you will probably have more than those to test, anyway. Good luck!